

COP 3515 – Advanced Program Design

Semester Programming Project

Client Change Request (CCR-006)

Project: Student Information Management System (SIMS)

Customer: University Registrar's Office

Priority: Medium

Assigned To: Student Developer

Sprint: Week 6

Due Date: Beginning of Lesson 14

Business Background

The University Registrar's Office is very pleased with the functionality that has been added to the Student Information Management System (SIMS). The software now allows employees to enter student information, record course grades, determine academic standing, save student records, and perform these tasks through an interactive menu.

However, the Registrar's Office has begun preparing for additional enhancements that will be requested later in the semester. During a review of the current source code, the University's Information Technology Department observed that many portions of the program contain similar or repeated code. As the application grows, duplicated code will become increasingly difficult to maintain because changes made in one location must also be made everywhere else.

The Registrar has requested that the software be reorganized so that each major task is performed by its own reusable function. The software should continue to behave exactly as it does today, but the internal organization should be significantly improved to make future enhancements easier to implement and maintain.

This enhancement should be added to the existing SIMS application developed during **CCR-001 through CCR-005. Do not create a new program.** Continue improving your existing codebase.

Business Requirements

The Registrar would like the application to continue performing **all functionality from CCR-001 through CCR-005**.

In addition, the software should be reorganized so that major operations are performed by separate functions.

At a minimum, the application should have separate functions that perform responsibilities such as:

- Displaying the program header
- Displaying the main menu
- Entering student information
- Entering course grades
- Calculating grade statistics
- Determining academic standing
- Displaying the student report
- Saving student information to a file
- Reading student information from a file (implemented)
- Processing menu selections

The customer expects the application to produce the **same results** as the previous version while making the software easier to maintain.

Repeated code should be eliminated whenever possible.

Acceptance Criteria

The project will be considered complete when all of the following requirements have been met.

- ☐ The program compiles without errors or warnings.
- ☐ All functionality from **CCR-001 through CCR-005** continues to work correctly.
- ☐ Major operations have been moved into separate functions.

- ☐ Functions perform one clearly defined task.
 - ☐ Duplicate code has been eliminated whenever practical.
 - ☐ Functions are called from main() rather than implementing all logic directly in main().
 - ☐ Program output matches previous project versions.
 - ☐ Program is easier to read and maintain.
-

Technical Constraints

For this week's assignment, you may **only** use C language features covered through **Lesson 14**.

Permitted concepts include:

- Variables
- Constants
- Arrays
- Enumerations
- Loops
- Selection statements
- File processing
- Functions
- Function parameters
- Function return values
- User input
- Formatted output

Do **not** use pointers or language features that have not yet been introduced in class.

Out of Scope

The Registrar is **not** requesting any of the following features at this time.

- Multiple students
- Pointer parameters
- Dynamic memory allocation
- Databases
- Editing student records
- Deleting student records
- Searching
- Graphical user interfaces (GUI)

These features will be requested in future Client Change Requests.

Questions for the Customer

Before writing any code, identify questions you would ask the customer to clarify the requirements.

Examples include:

1. Which operations should become separate functions?
2. Is there a preferred naming convention for functions?
3. Should every menu option have its own function?
4. Should calculations be placed in separate functions from input and output?
5. How small should individual functions be?
6. Should functions perform both input and output, or should those responsibilities be separated?
7. Is maintaining the existing user interface more important than changing the internal design?
8. Should helper functions be created even if they are only called once?
9. Are there coding standards that should be followed for function organization?
10. Should the program behavior remain identical after refactoring?
11. Will future versions require additional functions?

12. Should documentation be updated to reflect the new program structure?

Example Program Execution

Program Startup

Student Information Management System

Version 6.0

1. Add Student
2. Display Student
3. Enter Grades
4. Save Student Record
5. Exit

Selection:

Example 1 – Add Student

User Input

Selection: 1

Student ID: 1234567

Student Name: Alice Johnson

Current GPA: 3.84

Expected Output

Student successfully added.

Example 2 – Enter Grades

User Input

Selection: 3

95

88

91

84

100

Expected Output

Grades successfully recorded.

Example 3 – Display Student

User Input

Selection: 2

Expected Output

Student Summary

Student ID : 1234567

Student Name : Alice Johnson

Current GPA : 3.84

Academic Standing : Honors

Course Grades

95

88

91

84

100

Average Grade : 91.60

Highest Grade : 100

Lowest Grade : 84

Example 4 – Save Student Record

User Input

Selection: 4

Expected Output

Student record successfully saved.

student_records.txt created.

Example 5 – Exit

User Input

Selection: 5

Expected Output

Thank you for using the

Student Information Management System.

Program terminated successfully.

Software Engineering Requirement

This Client Change Request focuses on **refactoring** rather than adding significant new functionality.

Refactoring is the process of improving the internal design of software **without changing its observable behavior**.

Your instructor will evaluate both:

- The correctness of your program.
- The organization and readability of your source code.

A well-organized program is easier to understand, debug, maintain, and extend.

Regression Testing Requirements

Because the Student Information Management System is developed incrementally, **all previously implemented functionality must continue to operate correctly** after the refactoring.

Your submission must demonstrate that:

- CCR-001 functionality still works.
- CCR-002 functionality still works.
- CCR-003 functionality still works.
- CCR-004 functionality still works.
- CCR-005 functionality still works.
- CCR-006 successfully reorganizes the software without changing program behavior.

At a minimum, include the following regression tests.

Test Description	Expected Result
1 Add a student	Student stored successfully
2 Enter grades	Grades recorded correctly
3 Calculate grade statistics	Average, highest, and lowest grades are correct

Test Description	Expected Result
4 Determine academic standing	Correct standing displayed
5 Display student information	Complete report displayed
6 Save student record	File created successfully
7 Invalid menu selection	Error message displayed
8 Exit program	Program terminates normally

For **each** regression test include:

- Test description
- Input
- Expected output
- Actual output
- Pass/Fail result

If a defect is discovered:

1. Document the defect.
 2. Correct the program.
 3. Repeat the affected test.
 4. Verify that previously working functionality continues to operate correctly.
-

AI Usage Policy

Artificial Intelligence tools (such as ChatGPT, GitHub Copilot, Claude, or Gemini) may be used during this assignment.

However, you remain responsible for:

- Understanding every function you submit.
- Testing every feature.
- Correcting any defects.
- Demonstrating that the refactored application behaves identically to previous versions.

Remember that good software engineering is measured not only by **what** the program does, but also by **how well it is organized**.

Submission Requirements

Submit a single ZIP file containing:

- main.c
- Any additional source (.c) files (if applicable)
- Any header (.h) files (if applicable)
- student_records.txt (generated by your program)

Also include a PDF containing:

- A simple function hierarchy or call diagram showing the relationships among your functions
 - Screenshots demonstrating successful execution of each menu option
 - Results of all required regression tests
 - A brief description of any bugs encountered and how they were corrected
 - Answers to the **Questions for the Customer**
 - Any assumptions made because the customer requirements were ambiguous
-

Looking Ahead

This is the **sixth** release of the Student Information Management System (SIMS).

While this Client Change Request adds very little new user-visible functionality, it introduces an important software engineering practice: **refactoring**. As software systems evolve, improving the internal organization of the code is essential for supporting future enhancements. In upcoming Client Change Requests, you will extend this application using pointers, dynamic memory allocation, and more advanced programming techniques. A modular design built around well-defined functions will make those future enhancements significantly easier to implement and maintain.

Remember that professional developers spend a considerable amount of time improving existing code—not just adding new features. A successful solution demonstrates that the software has become **cleaner, more modular, and easier to maintain** while preserving all previously delivered functionality through comprehensive regression testing.